Assignment 2: Jacobians

16-384: Robot Kinematics and Dynamics

Prof. Jeff Ichnowski
Albert Xu
Henry Liao
Karthik Srinivasan

Contents

1 Theory Questions **3**

2 Code Questions **9**

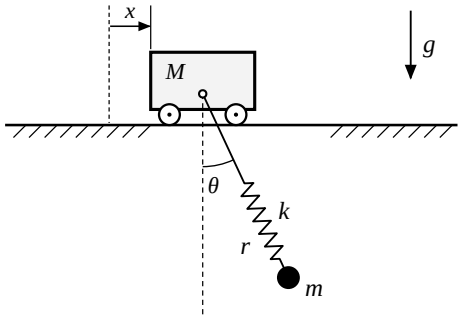
3 Hands-on Lab: Drawing with the xArm7 **11**
3.1 The xArm7 as a Planar Arm 11
3.2 Setting Up 11

4 Submission Checklist **15**

1 Theory Questions

1) Spring, cart, pendulum *30 points*

The following spring-cart-pendulum system shown below is a modification of the pendulum-cart discussed in class. The cart has mass M, the pendulum has mass m and its spring has rest length r_0 and spring constant k.



a. [10 points] Write the potential and kinetic energy of the system in terms of the generalized coordinates $q = [x, r, \theta]^\top$ and their time derivatives $\dot{q} = [\dot{x}, \dot{r}, \dot{\theta}]^\top$. Then write the Lagrangian of the system.

b. [10 points] Write the derivatives of $\mathcal{L}$ with respect to q and $\dot{q}$ (6 in total).

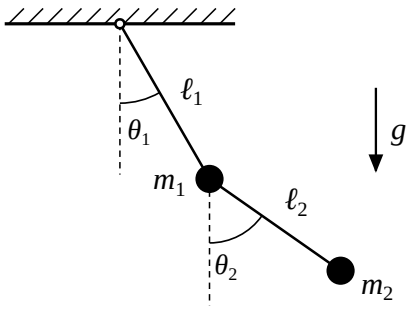
c. [5 points] Write the equations of motion of the system using the Euler-Lagrange equations.

d. [5 points] Suppose you can control the cart with an external force f_x. Write the equations of motion with the control input f_x.

2) Double Pendulum *30 points*

The double pendulum is a classic example of a chaotic system, yet its equations of motion are relatively simple to derive. You will be using your derived equations of motion to simulate the double pendulum in the coding section of this assignment.

The masses m_1 and m_2 are point masses at the ends of massless rods of length ℓ_1 and ℓ_2. Both angles are measured from the downward vertical, as shown in the figure: θ_2 is the absolute angle of link 2, *not* its angle relative to link 1. Gravity points in the $-y$ direction.



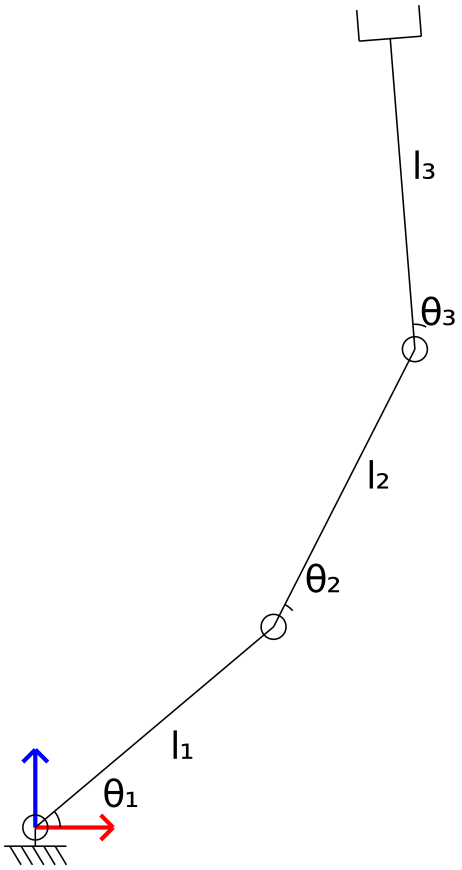
a. [10 points] Write the potential and kinetic energy of the system in terms of the generalized coordinates $q = [\theta_1, \theta_2]^\top$ and their time derivatives $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2]^\top$.

b. [10 points] Write the derivatives of $\mathcal{L}$ with respect to q and $\dot{q}$ (4 in total).

c. [10 points] Write the equations of motion of the system using the Euler-Lagrange equations.

3) RRR Arm Dynamics *40 points*

Consider the following robot. Assume no joint limits, and that the links lengths are $l_1 = l_2 = l_3 = 1$, and the masses are $m_1 = m_2 = m_3 = 1$. The joint angles are measured as a relative angle from the previous link / horizontal, so $\theta_1 = \theta_2 = \theta_3 = 0$ means the arm is fully extended to the right. Each link's mass is spread uniformly along its length, so its center of mass is at its midpoint and its moment of inertia about its center of mass is $I_i = \frac{1}{12} m_i l_i^2 = \frac{1}{12}$. Gravity points in the $-y$ direction.



a. [10 points] Write the potential and kinetic energy of the system in terms of the generalized coordinates $q = [\theta_1, \theta_2, \theta_3]^\top$ and their time derivatives $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2, \dot{\theta}_3]^\top$.

b. [10 points] Write the derivatives of $\mathcal{L}$ with respect to q and $\dot{q}$ (6 in total).

c. [10 points] Write the equations of motion of the system using the Euler-Lagrange equations. Include external torques τ_1, τ_2, τ_3 applied at each joint.

d. [5 points] Find the standard form matrices $M(q)$, $C(q, \dot{q})$, and $G(q)$ such that the equations of motion can be written as $M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q) = \tau$.

e. [5 points] What are the joint torques τ_1, τ_2, τ_3 required to hold the arm in the configuration $\theta_1 = \theta_2 = \theta_3 = 0$ (fully extended to the right) with no motion? How does this compare to the Jacobian-based gravity compensation method discussed previously?

2 Code Questions

4) Double Pendulum Simulation *45 points*

In this question you will simulate the double pendulum from the written section using the equations of motion you derived. All the code is in the `code/` folder; `code/README.md` explains how to run each script. You edit and submit only `double_pendulum.py`.

Use the same conventions as the written section: $q = [\theta_1, \theta_2]^\top$ with both angles measured from the downward vertical (counterclockwise positive), and point masses m_1, m_2 on massless links ℓ_1, ℓ_2. There is no applied torque in the code ($\tau = 0$). In your code, use the constants `M1`, `M2`, `L1`, `L2` and `G` at the top of `double_pendulum.py` rather than writing the numbers in directly, because the autograder changes them.

a. [20 points] Implement `dynamics(q, qd)`, which returns the joint accelerations $\ddot{q}$ as an array of shape $(2,)$. Your Euler-Lagrange equations are linear in $\ddot{q}$, so write them as $M(q)\ddot{q} = b(q, \dot{q})$ and solve for $\ddot{q}$ with `np.linalg.solve`.

b. [15 points] Implement `simulate(q0, qd0, T, dt)`. We recommend using a library like `scipy.integrate.solve_ivp` to integrate the equations of motion with the supplied `RTOL` and `ATOL` tolerances, but also encourage you to try implementing your own integrator to learn a little about numerical integration. You'll see the difference in (c).

The output format should be a tuple `(t, q, qd)` of numpy arrays, sampled every `dt` seconds from 0 to T inclusive (use `np.arange(0, T + dt / 2, dt)`, so there are $N = T/\Delta t + 1$ samples):

- `t`: the sample times $0, \Delta t, 2\Delta t, \ldots, T$, shape $(N,)$.
- `q`: the joint angles $[\theta_1, \theta_2]$ at each sample time, shape $(N, 2)$.
- `qd`: the joint velocities $[\dot{\theta}_1, \dot{\theta}_2]$ at each sample time, shape $(N, 2)$.

The first row of `q` and `qd` is the initial state `q0`, `qd0`.

Run `python local_autograder.py` to check parts (a) and (b), then watch your pendulum with `python visualize.py`.

c. [5 points] With no friction and no applied torque, the total energy $E = T + V$ of the double pendulum stays constant. That gives you a way to check a simulation without knowing the right answer: if E changes over time, the simulation is not exactly following the physics.

Implement `total_energy(q, qd)` in `double_pendulum.py` using your T and V from the written section (the autograder checks it too). Then run `python energy_check.py`, which simulates the pendulum for 20 seconds and plots its total energy over time.

Does your simulation conserve energy? If the energy changes noticeably, what could cause it?

d. [5 points] Run `python visualize.py --chaos 5 --plot`, which simulates five pendulums whose starting θ_2 values differ by only 0.001 rad. About how long do the pendulums stay together before they visibly separate? Now try a small initial angle, `python visualize.py --chaos 5 --q0 10 5`. Is the motion still chaotic? Explain the difference.

3 Hands-on Lab: Drawing with the xArm7

In this section the xArm7 holds a marker and writes on a sheet of paper. You already have most of the pieces: your Assignment 4 `Robot` does the inverse kinematics, and the arm is driven exactly as you drove planar chains there. What is new is where the paper is. You will find it the way a technician would: touch the marker to its four corners, and turn those four points into a coordinate frame for the sheet. Then you will draw the letters **KDC** on screen, run them in simulation, and finally watch the real arm write them.

3.1 The xArm7 as a Planar Arm

Hold joints 3 and 5 at zero and every joint of the xArm7 lies in one vertical plane, the one joint 1 turns. Joints 2, 4 and 6 are then three parallel hinges in that plane: a planar RRR arm standing up, with the marker on the end of its third link. To put the marker tip on a point $\mathbf{p} = [p_x, p_y, p_z]^\top$ in the robot's base frame,

1. joint 1 turns to face it, $q_1 = \operatorname{atan2}(p_y, p_x)$;
2. in that plane, $\mathbf{p}$ is at $r = \sqrt{p_x^2 + p_y^2}$ out and $z = p_z - 0.267$ m up from the shoulder;
3. your `Robot.inverse_kinematics` solves the RRR for the goal $[r, z, \phi]$, where ϕ is the angle of the third link that keeps the marker pointing into the paper.

`lab/drawing_arm.py` does steps 1 and 2 and calls your IK for step 3. It measures the three link lengths and the joint offsets from UFACTORY's own model of the arm, so you don't have to. Your IK takes 100 steps of size 0.1, which was sized for the metre-long links of Assignment 4. The xArm's links are about a third of that, so one call only gets part of the way, and `drawing_arm.py` calls it again from its own answer until the marker lands within 1 mm.

3.2 Setting Up

The lab uses the course robot library, `xarm7_lib`, the same one as the Assignment 3 lab. Update it before you start:

```
git clone https://github.com/CMU-16384-2026-Fall/16384-robot-lib
cd 16384-robot-lib
git pull                          # if you already had it cloned
conda env update -f environment.yml
conda activate 16384
```

Run every command in this section from the `lab` folder. As before, the same scripts drive the real arm when the `ROBOT_IP` environment variable is set, and a MuJoCo simulation, shown in your browser, when it is not. Everything up to the last part can be done at home.

5) Drawing KDC on the xArm7 *50 points*

a. [10 points] **The paper's frame.** Calibration gives you four touched corners $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4$ in the robot's base frame, in the order of Figure 1. They will not lie exactly on one plane: the paper isn't perfectly flat, and your hand isn't perfectly steady.

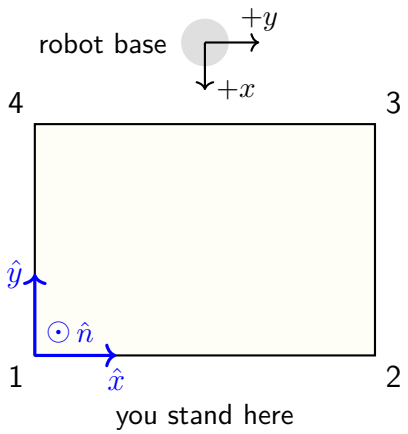


Figure 1: The sheet from above. You touch corners 1–4 in order; the paper frame has its origin at corner 1, $\hat{x}$ towards corner 2, and $\hat{n}$ out of the paper.

(i) Show that $\mathbf{n} = (\mathbf{p}_3 - \mathbf{p}_1) \times (\mathbf{p}_4 - \mathbf{p}_2)$ is perpendicular to both diagonals of the sheet, and explain why, when the four corners are coplanar, it is normal to the sheet. With corners ordered as in Figure 1, which way does it point?

(ii) You could instead take the normal from three corners, for example $(\mathbf{p}_2 - \mathbf{p}_1) \times (\mathbf{p}_4 - \mathbf{p}_1)$. Why is the diagonal version better for four noisy touches?

(iii) Let $\hat{n} = \mathbf{n}/\|\mathbf{n}\|$ and let $\mathbf{c} = \frac{1}{4}\sum_i \mathbf{p}_i$ be the centroid. Write formulas for the origin $\mathbf{o}$ (corner 1, moved along $\hat{n}$ onto the plane through $\mathbf{c}$), the unit x axis $\hat{x}$ (from corner 1 towards corner 2, within the plane), and $\hat{y}$. Assemble them into the 4×4 homogeneous transform $\mathbf{H}$ of the paper frame in the base frame.

(iv) A point (u, v) on the paper, at height h above it, is at $\mathbf{H}\,[u, v, h, 1]^\top$ in the base frame. Write the inverse map, from a base-frame point back to (u, v, h), without a matrix inverse.

b. [10 points] **Code it.** Implement your formulas in `lab/plane.py`: `plane_frame`, `paper_to_robot` and `robot_to_paper`. These three are autograded, on randomly perturbed corners as in Question 4. Then check them before going anywhere near the arm:

`python check_plane.py`

It tries your frame on sheets whose answer is known (flat, turned, tilted, a whiteboard standing up), on noisy touches checked against every property your formula promises, and on the graded cases, the same ones `local_autograder.py -p plane` runs from `code/`. Every failure says what is wrong. A wrong frame sends the marker into the table, so do not take the lab past this point until it reports no failures.

c. [10 points] **Draw it, in simulation.**

(i) Copy your `Robot` class from Assignment 4 into `lab/robot.py`. You don't need to change anything: the forward kinematics, Jacobian and gradient-descent IK you already wrote are what the arm runs.

(ii) Calibrate the simulator's sheet: `python calibrate.py`. The simulator has a sheet of paper of its own, placed a little crooked on purpose. It touches the four corners with your IK, builds the frame with your `plane.py`, and tells you how far your frame is from the sheet's true one. It should be within a couple of millimetres.

(iii) Draw the strokes for the letters **KDC**: `python sketch.py strokes/kdc.json`. Left-click to place points, right-click to finish a stroke; the window lists the other keys. Each stroke is traced without lifting the marker, so think about where to put the breaks. Closing the window saves the file. The strokes are stored in metres on the sheet, from corner 1, so the same file draws in simulation and on the real arm.

(iv) Run it: `python draw.py strokes/kdc.json`. Your IK is solved along the whole path first, which takes a few seconds, and any point it can't reach stops the run before anything moves. Then the simulated arm writes on its sheet, and ink appears where the marker meets the paper. The result is saved as `results/kdc-sim.png`: your strokes, dashed, beside the path the marker really took.

Include `results/kdc-sim.png` in your PDF. The ink sits a millimetre or two from your strokes. Why, when the arm follows its plan to a fraction of a millimetre?

d. [20 points] **Draw it on the xArm7.** Graded by hand, from a video and the result plot. In lab, with `ROBOT_IP` set:

(i) `git pull` your code onto the lab machine, and run `python check_plane.py` once more.

(ii) `python calibrate.py`. The arm moves to its ready pose, marker down, over the table. Then, for each corner in turn, it goes limp: push the marker tip onto the corner it asks for, keep the marker upright, hold it there gently, and press enter. Joints 3 and 5 stay held, so the arm stays in the plane its IK works in. If you push one of them out of place, the arm stops, asks you to let go, and puts it back.

(iii) `python draw.py strokes/kdc.json --plan-only` plans the drawing on your real sheet without moving anything. Look at `results/kdc-plan.png`.

(iv) `python draw.py strokes/kdc.json`, hand on the e-stop.

Safety: Read this before you touch the robot

- **The e-stop.** Know where it is before you start, and keep a hand on it while the arm draws. Press it for anything you did not expect. Stopping the arm costs you nothing.
- **Stay out of the workspace** while the arm moves under its own power. The only time you touch it is during calibration, when the script says the arm is free.
- **"Press enter when your hands are clear" means it.** Every move the scripts make on their own waits for that prompt.
- **The safety box** is the one from Assignment 3: $x \in [-0.10, 0.70]$, $y \in [-0.45, 0.45]$, $z \in [0, 0.80]$ m. `draw.py` checks the whole drawing against it, and against self-collision, before anything moves. It does *not* know about the marker, so the marker is yours to keep off the table: that is what the calibration is for.
- **If the marker digs in**, press the e-stop. The drawing presses 1 mm below the surface you touched, which a sprung marker holder absorbs. More than that means the calibration is off: calibrate again.

Deliverable. One video, a phone is fine, showing the arm write KDC from start to finish, with the paper visible. Put it in `lab/handin/`. `create_submission.py` also collects your `lab/robot.py`, `lab/plane.py`, your strokes, both calibrations and everything in `lab/results`. Include `results/kdc-robot.png` in your PDF, and say in a sentence or two where the real drawing differs from your strokes, and why.

4 Submission Checklist

- ☐ Create a PDF of your answers to the written questions with the name `<andrew_id>.pdf` and submit it to Gradescope.
- ☐ Include `lab/results/kdc-sim.png` and `lab/results/kdc-robot.png` in that PDF.
- ☐ Run `local_autograder.py` and `lab/check_plane.py`, and confirm they report no failures.
- ☐ Put your drawing video in `lab/handin`.
- ☐ Run `create_submission.py` in the python terminal.
- ☐ Rename `submission.zip` and upload `<andrew_id>.zip` to Gradescope.
- ☐ Coding questions are autograded, with no limit on number of submissions within the mandated deadline.